RETRIEVAL-AUGMENTED GENERATION

RAG Pipeline — Comprehensive Training Guide

End-to-End Pipeline . Naive RAG . Limitations & Mitigations

Advanced AI Training Series | LLMOps & Agentic AI Module

Confidential — Internal Training Material

1. What is Retrieval-Augmented Generation (RAG)?

Retrieval-Augmented Generation (RAG) is an architectural pattern that enhances Large Language Model (LLM) responses by dynamically injecting **relevant, external knowledge** into the prompt at inference time. Rather than relying solely on knowledge baked into model weights during training, RAG retrieves factual evidence from a curated knowledge base and provides it as context to the generator.

Originally proposed by Lewis et al. (2020, Meta AI), RAG solves the fundamental tension between the **static nature of pre-trained LLMs** and the **dynamic, proprietary nature of real-world information**. It has become the dominant grounding technique in production LLM systems.

RAG vs Fine-Tuning vs Prompt Engineering — When to Use What

Approach	Mechanism	Knowledge Update	Cost	Best For
Prompt Engineering	Instructions in prompt	Manual re-prompt	Very Low	Simple tasks, no custom knowledge
RAG	Retrieve at inference time	Add/update docs in DB	Low–Medium	Dynamic, large, or proprietary knowledge bases
Fine-Tuning	Retrain on domain data	Retrain required	High	Style/behavior change, stable domain knowledge
RAG + Fine-Tuning	Both combined	Update docs; retrain for style	High	Enterprise: accuracy + tone control

2. RAG End-to-End Pipeline: Chunk → Embed → Store → Retrieve → Generate

The RAG pipeline is divided into two phases: the **Indexing Phase** (offline, run once or periodically) and the **Inference Phase** (online, run per query). Together they form a five-stage pipeline.

Step	Stage	Description
1	CHUNK	Split source documents into smaller, semantically coherent text segments (chunks). Controls context window usage and retrieval precision.
2	EMBED	Convert each chunk into a dense numerical vector using an embedding model (e.g., text-embedding-3-small, BGE, E5). Captures semantic meaning.
3	STORE	Persist vectors in a Vector Database (e.g., FAISS, Chroma, Pinecone, Weaviate, pgvector) alongside chunk metadata for fast similarity lookup.
4	RETRIEVE	At query time, embed the user question and perform Approximate Nearest Neighbour (ANN) search to fetch the top-k most relevant chunks.
5	GENERATE	Provide the retrieved chunks as context in the LLM prompt. The model synthesises a grounded, factual answer using only the supplied context.

2.1 Indexing Phase (Offline)

Stage 1 — Document Chunking

Chunking converts raw source documents (PDFs, HTMLs, DOCX, markdown) into segments small enough to fit in the LLM's context window while large enough to be semantically complete. The chunk size directly affects both retrieval precision and generation quality.

Strategy	How It Works	Chunk Size	Pros	Cons
Fixed-Size	Split every N tokens, optional overlap	256–512 tokens	Simple, predictable	Breaks sentences mid-way
Sentence Splitter	Split on sentence boundaries	1–5 sentences	Preserves sentences	Variable size
Recursive Character	Hierarchical split: \n\n → \n → space	Configurable	Best for mixed text	Slightly complex
Semantic Chunking	Embed sentences; split on semantic shift	Dynamic	Preserves meaning	Slower, needs embeddings
Parent-Child	Small retrieval chunks, large synthesis chunks	Child: 128 tok	Precision + context	Dual-index needed
Document-based	Split on headings/sections	Varies	Structure-aware	Requires structured docs

Key Parameters:

- **chunk_size**: Number of tokens per chunk (typical: 256–512 for retrieval, 1024 for synthesis)
- **chunk_overlap**: Token overlap between consecutive chunks (typical: 20–50 tokens) to avoid context loss at boundaries
- **separators**: Priority list of split characters (e.g., paragraph → sentence → word)

```
# LangChain Recursive Character Text Splitter from langchain.text_splitter import RecursiveCharacterTextSplitter splitter = RecursiveCharacterTextSplitter( chunk_size=512, chunk_overlap=50, separators=['\n\n', '\n', '. ', ' '] ) chunks = splitter.split_documents(documents)
```

Stage 2 — Embedding

Each chunk is passed through an **embedding model** to produce a fixed-size dense vector (typically 384–3072 dimensions). These vectors encode the semantic meaning of the text in a high-dimensional space where similar meanings cluster together (cosine similarity). The choice of embedding model is the single largest determinant of RAG retrieval quality.

Model	Provider	Dimensions	Max Tokens	Use Case
text-embedding-3-small	OpenAI	1536	8191	Cost-effective production
text-embedding-3-large	OpenAI	3072	8191	High-accuracy tasks
text-embedding-ada-002	OpenAI	1536	8191	Legacy / compatibility
BAAI/bge-large-en-v1.5	HuggingFace (BAAI)	1024	512	Open-source, on-premise
intfloat/e5-large-v2	HuggingFace	1024	512	Strong multilingual
voyage-large-2	Voyage AI	1536	16000	Long documents
embed-english-v3.0	Cohere	1024	512	Enterprise API

```
# OpenAI Embedding Example from openai import OpenAI client = OpenAI() response = client.embeddings.create( input=[chunk.page_content for chunk in chunks], model='text-embedding-3-small' ) vectors = [r.embedding for r in response.data]
```

Stage 3 — Vector Storage

Embeddings and their associated metadata (source, page number, chunk text) are stored in a **Vector Database**. Unlike traditional relational databases, vector DBs are optimised for *Approximate Nearest Neighbour (ANN) search* using algorithms such as HNSW (Hierarchical Navigable Small World graphs) and IVF (Inverted File Index). The choice of vector DB depends on scale, deployment model, and latency requirements.

Vector DB	Deployment	Index Type	Scale	Standout Feature
FAISS	Local / in-memory	IVF, HNSW, Flat	Millions	Ultra-fast, Meta, open-source

Vector DB	Deployment	Index Type	Scale	Standout Feature
Chroma	Local / server	HNSW (hnswlib)	Thousands–Millions	Developer-friendly, LangChain native
Pinecone	Managed cloud	Proprietary (pod/serverless)	Billions	Fully managed, production SLA
Weaviate	Self-hosted / cloud	HNSW	Billions	GraphQL API, multi-modal
Qdrant	Self-hosted / cloud	HNSW	Billions	Filtering + ANN combined
pgvector	PostgreSQL extension	IVF Flat / HNSW	Millions	SQL + vectors in one DB
Azure AI Search	Managed cloud (Azure)	HNSW	Billions	Azure ecosystem integration
OpenSearch	Self-hosted / AWS	HNSW, IVF	Billions	Hybrid BM25 + vector

2.2 Inference Phase (Online / Per Query)

Stage 4 — Retrieval

At query time, the user's question is embedded using the **same embedding model** used during indexing. The resulting query vector is compared against all stored chunk vectors using a similarity metric (typically cosine similarity or dot product). The top-k most similar chunks are returned as context.

Retrieval Method	Mechanism	Strength	Weakness
Dense Retrieval (ANN)	Cosine similarity on embeddings	Semantic understanding	Misses exact keyword matches
Sparse Retrieval (BM25)	TF-IDF keyword frequency scoring	Precise keyword match	No semantic understanding
Hybrid Retrieval	Combines dense + sparse scores (RRF)	Best of both worlds	More complex pipeline
MMR (Max Marginal Relevance)	Balances relevance and diversity	Reduces redundancy	Slower, needs tuning
Multi-Query Retrieval	LLM generates multiple query variants	Better coverage	Extra LLM calls, cost
HyDE (Hypothetical Doc)	LLM generates hypothetical doc, embed it	Bridges query-doc gap	Risk of hallucinated context

Retrieval Quality Metrics:

Metric	Formula / Definition	Target
Recall@k	Fraction of relevant docs in top-k results	> 0.85
Precision@k	Fraction of top-k results that are relevant	> 0.75
MRR (Mean Reciprocal Rank)	Average of 1/rank of first relevant result	> 0.80
NDCG@k	Normalized Discounted Cumulative Gain at k	> 0.80
Hit Rate	Does the correct chunk appear in top-k?	> 0.90

Stage 5 — Generation

The retrieved chunks are assembled into a **prompt template** and passed to the LLM. The model is instructed to answer strictly based on the provided context, reducing hallucination. Generation quality depends on prompt design, context ordering, and the generator model itself.

```
# Typical RAG Prompt Template SYSTEM = ''' You are a helpful assistant. Answer ONLY using the context below. If the answer is not in the context, say 'I don't know.' Context: {context} ''' # Assemble context from retrieved chunks context = '\n\n---\n\n'.join([doc.page_content for doc in retrieved_docs]) prompt = SYSTEM.format(context=context) + f'\nQuestion: {query}'
```

Prompt Component	Purpose	Best Practice
System Instruction	Define role and answer boundaries	Explicitly instruct: 'Use ONLY the provided context'
Context Block	Inject retrieved chunks	Separate with '---', order by relevance DESC, include source metadata
Query	User question	Include verbatim; optionally rewrite for clarity
Output Format	Control response structure	Specify: JSON, bullet points, paragraph as needed
Citation Instruction	Ground answer to sources	Ask model to cite [Source: filename] per claim

3. Naïve RAG — Basic Retrieval-Based QA

Naïve RAG is the simplest, most straightforward implementation of the RAG paradigm. It implements the five-stage pipeline with minimal complexity: fixed-size chunking, a single embedding model, a flat vector index, single-query top-k retrieval, and a basic stuffed prompt. It is the **baseline architecture** used to validate RAG feasibility on a new domain.

3.1 Naïve RAG Architecture Overview

Component	Typical Choice in Naïve RAG	Configuration
Document Loader	LangChain PyPDFLoader / DirectoryLoader	Load .pdf, .txt, .md files
Text Splitter	RecursiveCharacterTextSplitter	chunk_size=500, chunk_overlap=50
Embedding Model	OpenAI text-embedding-3-small	Default dimensions: 1536
Vector Store	FAISS (in-memory) or Chroma	Flat L2 or cosine index, no filtering
Retriever	VectorStoreRetriever (k=3 or 4)	Single query, cosine similarity
LLM	gpt-3.5-turbo or gpt-4o-mini	Temperature=0 for factual tasks
Prompt Template	Simple stuffing: context + question	No few-shot, no CoT
Chain	RetrievalQA (LangChain) or LCEL	Single-pass: retrieve then generate

3.2 Naïve RAG — Full Code Walkthrough

```
# ■■ Naïve RAG with LangChain ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ from
langchain_community.document_loaders import PyPDFLoader from langchain.text_splitter import
RecursiveCharacterTextSplitter from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS from langchain.chains import RetrievalQA
# Step 1 — Load Documents loader = PyPDFLoader('knowledge_base.pdf') documents =
loader.load() # Step 2 — Chunk splitter = RecursiveCharacterTextSplitter(chunk_size=500,
chunk_overlap=50) chunks = splitter.split_documents(documents) # Step 3 — Embed + Store
embeddings = OpenAIEmbeddings(model='text-embedding-3-small') vectorstore =
FAISS.from_documents(chunks, embeddings) # Step 4 — Retriever retriever =
vectorstore.as_retriever(search_kwargs={'k': 4}) # Step 5 — Generate llm =
ChatOpenAI(model='gpt-4o-mini', temperature=0) qa_chain = RetrievalQA.from_chain_type(
llm=llm, retriever=retriever, return_source_documents=True ) result =
qa_chain.invoke({'query': 'What is the refund policy?'}) print(result['result'])
```

3.3 What Naïve RAG Does Well

✔ Strengths of Naïve RAG

- Fast to prototype — a working system in under 50 lines of code
- Effective on focused, well-structured knowledge bases (e.g., product manuals, policy documents)
- Low infrastructure cost — FAISS runs entirely in-memory
- Transparent retrieval — easy to debug which chunks were retrieved
- Sufficient for QA over small corpora (< 10,000 documents)
- Good baseline for benchmarking advanced RAG improvements

4. Naïve RAG — Limitations

While Naïve RAG is a powerful starting point, it suffers from several systematic limitations that manifest in production systems. These can be grouped into **Retrieval-side failures**, **Generation-side failures**, and **Infrastructure / scalability constraints**.

4.1 Retrieval-Side Limitations

Limitation	Root Cause	Observable Symptom	Mitigation Strategy
Semantic Gap	Query phrasing differs from document phrasing even when meaning is the same	Correct chunks scored low; irrelevant chunks ranked high	HyDE, query rewriting, multi-query retrieval
Chunk Boundary Loss	Fixed-size splitting cuts critical information across chunk boundaries	Retrieved chunk lacks essential context (e.g., answer split across two chunks)	Overlap, sentence-aware splitting, parent-child chunking
Missing Global Context	Individual chunks lack document-level or structural context	Model cannot answer questions requiring synthesis across sections	Document summarisation, hierarchical indexing, MapReduce chain
Top-k Hard Cutoff	Relevance threshold is fixed; k=4 may miss the 5th crucial chunk	Incomplete answers; multi-part questions answered partially	Dynamic k, MMR, score-threshold filtering
Retrieval of Redundant Chunks	Near-duplicate sentences stored as separate chunks	Context window wasted on repeated information	Deduplication, MMR diversity, chunk-level dedup at index time
Keyword Blindness	Dense retrieval ignores exact keyword/entity matches	Named entities, product codes, IDs not found reliably	Hybrid retrieval: BM25 + dense with Reciprocal Rank Fusion (RRF)
Multi-hop Queries	Retriever fetches only directly relevant chunks, missing intermediate evidence	Answers to complex reasoning chains are incorrect or incomplete	Iterative retrieval, graph RAG, chain-of-thought retrieval
Outdated Index	New documents added to source without re-indexing	System answers with stale information	Incremental indexing, document versioning, TTL-based refresh

4.2 Generation-Side Limitations

Limitation	Root Cause	Observable Symptom	Mitigation Strategy
Hallucination Despite Retrieval	LLM ignores provided context or blends context with training knowledge	Factually incorrect answers even when correct chunk was retrieved	Strict system prompt, faithfulness evaluation (RAGAs), RLHF
Context Window Overflow	Total tokens (system + context + query) exceed model context limit	Truncation of important chunks; API errors	Chunk size tuning, reranking + top-k reduction, summarise-then-retrieve
Lost in the Middle	LLMs attend poorly to chunks placed in the middle of the context	Information in the middle of the prompt is ignored	Place most relevant chunks first and last; reranking
Answer Abstraction Failure	Model answers at chunk level, not question level	Response is verbatim copy of chunk, not a synthesised answer	Better prompt engineering, abstractive summarisation step
Conflicting Retrieved Chunks	Different chunks contain contradictory information	Model produces hedging, inconsistent, or confused answers	Temporal metadata filtering, source authority ranking
Language / Format Mismatch	Retrieved chunks are in a different language or format than the query	Translated or misformatted answers	Language filtering at retrieval, multilingual embeddings

4.3 Infrastructure and Scalability Limitations

Limitation	Impact	At Scale Threshold	Solution Path
FAISS In-Memory Only	Index lost on restart; cannot exceed RAM	> 1M vectors or 4GB RAM	Migrate to Chroma (persistent), Qdrant, or Pinecone
No Metadata Filtering	Cannot restrict retrieval to subset of documents	Multi-tenant or date-filtered queries	Qdrant/Weaviate payload filters, LangChain SelfQueryRetriever
No Access Control	All users access all indexed documents	Enterprise multi-user deployment	Namespace isolation, row-level security in pgvector
Embedding Cost	Cost scales linearly with document count and updates	> 100K chunks re-indexed frequently	Batch embedding, caching, lightweight local models (BGE)
No Observability	No logging of queries, retrieved chunks, or answer quality	Day 1 in production	LangSmith, Arize Phoenix, Ragas, custom logging middleware
Lack of Reranking	ANN retrieval rank != relevance rank for generation	Any production system	Cross-encoder reranker: Cohere rerank, BGE-reranker, Flashrank

4.4 Evaluation Gap in Naïve RAG

One of the most overlooked limitations of Naïve RAG is the absence of systematic evaluation. Without measurement, teams cannot distinguish retrieval failures from generation failures, making iterative improvement impossible. The RAGAS framework provides component-level metrics.

RAGAS Metric	What It Measures	Evaluates	Score Range
Faithfulness	Is the answer grounded in the retrieved context? Are all claims supported?	Generation quality	0.0 – 1.0
Answer Relevance	Does the answer actually address the user's question?	Generation quality	0.0 – 1.0
Context Precision	What fraction of retrieved chunks were actually relevant to the question?	Retrieval quality	0.0 – 1.0
Context Recall	Were all relevant chunks successfully retrieved?	Retrieval quality	0.0 – 1.0
Context Relevance	How relevant is the retrieved context to the query on average?	Retrieval quality	0.0 – 1.0
Answer Correctness	Does the answer match ground-truth? (factual F1 + semantic similarity)	End-to-end	0.0 – 1.0

5. Evolutionary Path: Naïve → Advanced → Modular RAG

The RAG research community has systematically addressed each limitation of Naïve RAG, producing a layered evolution of progressively sophisticated architectures.

Generation	Architecture	Key Additions over Naïve RAG	Typical Use Case
Gen 1	Naïve RAG	Fixed chunking, single embedding, flat ANN, simple stuffing prompt	Prototyping, small focused knowledge bases
Gen 2	Advanced RAG	Query rewriting, hybrid retrieval (BM25+dense), reranking (cross-encoder), sentence-window retrieval, metadata filtering, iterative retrieval	Production QA, enterprise search, chatbots
Gen 3	Modular RAG	Pluggable modules: routing, fusion, adaptive retrieval, FLARE, Self-RAG, corrective RAG (CRAG), knowledge graph integration	Complex reasoning, multi-domain, agentic systems
Gen 4	Graph RAG	Knowledge graph extraction from documents; entity-relationship traversal at query time; community summarisation for global queries	Research analysis, legal, financial intelligence
Gen 5	Agentic RAG	LLM decides when and what to retrieve; multi-step tool use; self-reflection and iterative refinement (ReAct, Plan-and-Execute)	Complex multi-hop tasks, autonomous research agents

6. Quick Reference — Common Failure Modes & Fixes

Symptom	Likely Cause	Diagnostic Step	Fix
Wrong answer, right topic	Chunk boundary cut key info	Print retrieved chunks	Increase chunk_overlap to 100+
Completely irrelevant answer	Retrieval failure; query not matching chunks	Check similarity scores	Hybrid retrieval + query rewrite
Answer says 'I don't know' but answer exists in docs	Chunk not indexed or k too small	Direct similarity search	Increase k; verify indexing pipeline
Contradictory answer	Multiple conflicting chunks retrieved	Log all retrieved chunks	Add date/version metadata filter
Answer ignores retrieved context	LLM relies on training knowledge	Test with known-false context	Stronger system prompt; temperature=0

Symptom	Likely Cause	Diagnostic Step	Fix
Slow retrieval (> 500ms)	FAISS flat index on large corpus	Profile retrieval step	Switch to HNSW index or Qdrant
High hallucination rate	LLM fabricates beyond context	RAGAS faithfulness < 0.7	Add 'cite your sources' instruction; RLHF
Poor multilingual performance	English-only embedding model	Test cross-lingual query	Use multilingual-e5 or LaBSE

■ Production Checklist Before Going Live with Any RAG System

- Evaluate retrieval quality: Context Recall > 0.85 on held-out question set
- Evaluate generation quality: Faithfulness > 0.85, Answer Relevance > 0.80 (RAGAS)
- Implement metadata filtering (date, source, tenant) to prevent cross-contamination
- Add a reranker (cross-encoder) between retrieval and generation stages
- Set up observability: log query, retrieved chunks, answer, latency for every request
- Test edge cases: out-of-domain queries, ambiguous questions, adversarial prompts
- Define a re-indexing schedule aligned with document update frequency
- Implement rate limiting and cost budgets on embedding and LLM API calls

7. Glossary of RAG Terms

Term	Definition
ANN (Approx. Nearest Neighbour)	Fast algorithm (HNSW, IVF) for finding vectors with highest similarity without exhaustive search
BM25	Probabilistic keyword-based retrieval algorithm used in Elasticsearch and Solr; foundation of hybrid retrieval
Chunk	A fixed-size or semantically coherent sub-segment of a source document used as the unit of retrieval
Chunk Overlap	Number of shared tokens between consecutive chunks to preserve boundary context
Context Window	Maximum token limit that an LLM can process in a single call (input + output)
Cross-Encoder Reranker	A model that jointly encodes query + chunk to produce a refined relevance score; more accurate than bi-encoder
Dense Retrieval	Vector similarity search using neural embedding models; captures semantic meaning
Embedding	A numerical vector representation of text capturing semantic meaning in a high-dimensional space
FAISS	Facebook AI Similarity Search — an open-source library for efficient ANN search
Faithfulness	RAGAS metric measuring whether all answer claims are grounded in retrieved context
FLARE	Forward-Looking Active Retrieval — model triggers retrieval only when uncertain
HyDE	Hypothetical Document Embeddings — generate a fake answer and embed it for retrieval
Hybrid Retrieval	Combination of sparse (BM25) and dense (ANN) retrieval with score fusion (RRF)
HNSW	Hierarchical Navigable Small World — graph-based ANN index offering excellent speed/accuracy tradeoff
Lost in the Middle	Empirical finding that LLMs ignore context placed in the middle of long prompts
MMR	Maximum Marginal Relevance — retrieval strategy balancing relevance and diversity
RAG	Retrieval-Augmented Generation — grounding LLM responses with external knowledge retrieval
RAGAS	RAG Assessment framework providing automated metrics: Faithfulness, Answer Relevance, Context Precision/Recall

Term	Definition
Reranking	Post-retrieval step that re-scores top-k chunks using a more powerful cross-encoder model
RRF	Reciprocal Rank Fusion — algorithm to merge ranked lists from multiple retrieval systems
Self-RAG	Model trained to decide when to retrieve, what to retrieve, and how to use retrieved text
Semantic Chunking	Splitting documents based on semantic similarity shifts rather than fixed token counts
Sparse Retrieval	Term-frequency based retrieval (BM25, TF-IDF); fast, interpretable, keyword-focused
Top-k	Number of document chunks returned by the retriever for inclusion in the prompt
Vector Database	Specialised database storing and indexing high-dimensional embeddings for ANN search

References: *Lewis et al. (2020) — Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Gao et al. (2023) — Retrieval-Augmented Generation for Large Language Models: A Survey. RAGAS: Automated Evaluation of RAG Pipelines (Es et al., 2023). Edge et al. (2024) — From Local to Global: A Graph RAG Approach to Query-Focused Summarization.*